Semantic Normalization for Binary Function Similarity Detection: You Don't Always Need a Larger Model

Anonymous Author(s)

***Abstract*—Binary Function Similarity Detection (BFSD) underpins security applications such as vulnerability search, malware analysis, and software component analysis, yet it remains challenging across architectures, compilers, and optimization settings because functions compiled from the same source code can look drastically different under different instruction sets, compilers, and optimization options. Recent research has increasingly relied on larger and larger models to learn across this variation. In this paper, we take a different approach: we propose a semantic normalization pipeline that lifts binaries from different architectures into LLVM Intermediate Representations (IRs) and applies IR-level re-optimization to reduce discrepancies that arise from compilation options and, at least in part, could have been removed before downstream tasks. The normalized output can both improve legacy binary-based BFSD systems through recompilation and serve as input to IR-native models. Building on this pipeline, we implement ReGraphv2, a lightweight IR-native model that combines sequential and structural information on normalized IR. Across a broad range of legacy and learning-based methods, semantic normalization consistently improves their accuracy, and a lightweight IR-native model built on the normalized IR performs strongly in large-scale retrieval and in cross-architecture vulnerability search on real-world firmware. More broadly, we hope this work draws the attention of BFSD research to normalizing binary functions before training, so that models can focus on program semantics rather than on differences that can be removed beforehand.**

***Index Terms*—Binary Code Similarity Detection, Code Lifting, Binary Code Re-Optimization.**

1. Introduction

Binary Function Similarity Detection (BFSD) is widely used for security tasks such as vulnerability search, malware analysis, patch identification, and software plagiarism detection [1–4]. In many of these scenarios, analysts have no access to source code, especially in legacy software, proprietary components, or third-party libraries that are released only as binaries [1, 2]. Security analysts therefore often have to examine binaries directly rather than source code. As a result, the ability to determine whether two binary functions implement the same logic remains fundamental to understanding library reuse and tracing vulnerable or malicious code across various systems [1, 3, 5].

Binaries derived from the same source code can differ substantially once they pass through different compilation pipelines [6, 7]. Architectural differences change the instruction set, register usage, and calling conventions, while compilers and optimization levels further reshape instruction selection, control flow, data layout, and even the visible structure of a function [7, 8]. Consequently, semantically equivalent functions may look very different in assembly, which renders traditional methods, such as fuzzy hashing and direct graph matching, often show poor performance in cross-architecture and cross-optimization settings [8].

Despite these challenges, learning-based methods have considerably improved BFSD [3, 4, 6, 9–11]. Researchers have explored graph neural networks (GNNs), transformer-based encoders, and, more recently, large language models (LLMs) as function encoders for binary similarity matching [3, 4, 6, 9, 12]. Yet a common pattern in recent work is to rely on increasingly large models to learn discrepancies introduced by compilation and thereby achieve stronger matching performance [4, 12–15]. This trend improves accuracy, but it also brings substantially higher computational overhead, making large scale function retrieval increasingly infeasible. In our view, rather than relying on increasingly large models to relearn every difference introduced by architectures and compilation settings, a more effective strategy is to mitigate removable discrepancies in advance, allowing the model to focus on the remaining program semantics. We therefore propose a semantic normalization pipeline for BFSD. Our pipeline lifts binaries from different architectures into LLVM IR, a common representation with richer semantics and mature optimization support [16, 17], and then applies IR-level re-optimization to reduce discrepancies introduced by different compiler optimization settings.

The resulting normalized IR is used in two ways. First, we regenerate binaries from normalized IR so that existing BFSD methods that operate only on binaries can also benefit from this normalization pipeline. Second, we build an IR-native model ReGraphv2 that works directly on the normalized IR. To the best of our knowledge, no publicly available model learns directly from the lifted IR for BFSD. Prior methods instead operate on assembly, decompiled pseudo-C, or program graphs. We therefore design an IR-native model that combines sequential token modeling with structural information from control-flow and data-dependence graphs. Across our benchmark, semantic normalization consistently improves a broad range of existing methods. Build upon normalized IR, ReGraphv2 achieves the best performance in

function retrieval from large candidate pools while remaining lighter than recent LLM alternatives, in public datasets and real-world firmware, outperforming the strongest baseline in most settings.

Our contributions can be summarized as follows:

- We analyze how compiler optimizations introduce variation into binaries, and we propose a semantic normalization pipeline that lifts binaries from different architectures into a common LLVM IR and re-optimizes it with a tailored level we call `-Oc`. Applied to a various of legacy BFSD methods, the pipeline substantially improves their accuracy, and by regenerating binaries from the normalized IR, it enables tools that are confined to a single architecture to operate across architectures.
- We build ReGraphv2, to our knowledge the first publicly available BFSD model trained directly on LLVM IR. It couples sequential token modeling over the IR with structural cues from control-flow and data-dependence graphs, outperforms the compared state-of-the-art methods in large-pool retrieval by a clear margin, and achieves the best performance on a real-world benchmark of cross-architecture vulnerability search in firmware images.
- We release our pipeline and trained models as open source[1]. With this pipeline, even a compact LLM runs on normalized IR can outperform an LLM with several times its size on raw assembly. This indicates that the normalized input lets a model concentrate its capacity on program semantics rather than on removable compilation differences. We hope this study offers new insight into where, in the BFSD pipeline, compilation-induced variation is best removed.

The remainder of the paper proceeds as follows. Section 2 motivates semantic normalization with a concrete example, and Section 3 situates our work among IRs and prior BFSD methods. We describe the normalization pipeline and the ReGraphv2 model in Section 4, with implementation details in Section 5. Section 6 presents our evaluation, and Section 7 examines threats to validity and future work before Section 8 concludes.

2. Motivation

In this section, we motivate our design through a concrete example. We show how lifting and re-optimization can systematically reduce discrepancies and thereby benefit downstream binary analysis tasks.

Consider the example C function shown in Code 1. The function performs two simple loops. The first loop accumulates the integers from 0 to 999 into the first argument `a`, and then applies a sequence of `XOR` operations from 0 to 4 to the second argument `b`. After that, the function returns their sum. We compile this function into two binaries, one for ARM64 with `-O0` and the other for x86 with `-O1`, and then trace how the two binaries evolve through our normalization pipeline[2].

Code 1: Example function source code in C.

```c
int example(int a, int b) {
    // First loop, accumulated sum.
    for(int i = 0; i < 1000; i++) {
        a += i;
    }
    // Second loop, accumulated XOR.
    for (int i = 0; i<5; i++) {
        b ^= i;
    }
    // Sum the results from the two loops.
    return a + b;
}
```

Raw Assembly: At the instruction level, the two binaries appear to have almost nothing in common. Different ISAs imply different opcodes, registers, and syntactic conventions, so the two disassemblies in Code 2 look like entirely unrelated texts even though they are compiled from the same source code.

Code 2: Assembly code of the example function compiled for ARM -O0 (left) and x86 -O1 (right).

```
SUB SP, SP, #0x10
STR W0, [SP, #0x10+var_4]
...
RET
```

```
XOR EAX, EAX
XOR ESI, EAX,
...
RETN
```

Lifted IR: Lifting both binaries to LLVM IR substantially reduces the ISA-level discrepancy. As shown in Code 3, the two functions now share a common instruction set and type system. What remains is primarily the variation introduced by different compiler optimization levels. As shown in Code 4 and Code 5, the ARM `-O0` version still contains both explicit loops, whereas the x86 `-O1` version has already folded the first loop into adding the constant 499500, the sum of the integers from 1 to 999[3].

Code 3: LLVM IR of the example function compiled for ARM -O0 (left) and x86 -O1 (right).

```
define i64 @"example"(i32 %"
    .1", i32 %".2")
{
"@2":
  %".13" = load i32, i32* %"
    i"
  %".14" = icmp sge i32 %"
    .13", 1000
  ...
```

```
define i64 @"example"(i32 %"
    .1", i32 %".2")
{
"@2":
  %".11" = load i32, i32* %"
    i"
  %".12" = load i32, i32* %"
    a2"
  ...
```

1. Source code and trained models will be released publicly. The repository link is withheld here to preserve anonymity during double-blind review.

2. For brevity, we show only parts of the assembly and IR in this section. The complete code is available in our repository.

3. For better display purpose, we decompile the lifted IR back into pseudo-C with IDA Pro, however, our pipeline itself operates directly on the IR only.

Code 4: Pseudo-C for the example function, ARM -O0.

```
__int64 __fastcall example(int a1, int a2)
{
  int j;
  int i;
  for ( i = 0; i < 1000; ++i )
    a1 += i;
  for ( j = 0; j < 5; ++j )
    a2 ^= j;
  return (a1 + a2);
}
```

Code 5: Pseudo-C for the example function, x86 -O1.

```
__int64 __fastcall example(int a1, int a2)
{
  int i;
  for ( i = 0; i != 5; ++i )
    a2 ^= i;
  return a1 + a2 + 499500;
}
```

Re-optimized IR: To further reduce the remaining optimization-induced discrepancies, we run both IRs through a uniform `-O3` optimization pipeline. The result is shown in Code 6. After re-optimization, the two functions collapse to the same closed form. The second loop is simplified to a single `XOR`, and the entire function becomes a compact expression. In other words, two binaries that looked unrelated at the assembly level become closely aligned at the re-optimized IR level.

Code 6: Pseudo-C for both ARM -O0 and x86 -O1 after uniform re-optimization.

```
int example(int a1, int a2)
{
  return a1 + (a2 ^ 4) + 499500;
}
```

This example motivates the main intuition behind our pipeline. Lifting mitigates architecture-induced differences, while re-optimization reduces discrepancies introduced by compiler optimizations. Together, these two stages remove a substantial portion of the variance that would otherwise be left for the similarity models to pay extra effort to learn, allowing the downstream methods to focus on the remaining semantic content rather than on those differences that should have been mitigated systematically in our pipeline.

3. Background and Related Works

In this section, we review the background most relevant to our approach, including intermediate representations, LLVM IR lifting systems, and prior studies on binary function similarity detection. We focus in particular on the properties that make LLVM IR suitable as a unified representation and on the limitations of existing lifting and modeling strategies that motivate our design choices.

3.1. Intermediate Representation

Intermediate representations serve as the bridge between source and binary programs for downstream tasks such as optimization, analysis, and code generation. In compilers, source code is lowered into IR during parsing, then the IR flows through optimization passes before final code generation. GCC, a widely-used C compiler, uses a multi-layer IR system comprising GENERIC, GIMPLE, and RTL [18, 19]. Clang, another popular C compiler, lowers source code to LLVM IR [17], which passes through a series of modular optimization and transformation passes [16].

In binary analysis, several projects have introduced their specific IRs. Angr [20] lifts binaries to an IR derived from VEX [21] for symbolic execution. BAP [22] proposes the BIL for formal verification of binary code. He et al. [3] design a custom IR that makes data and control dependencies explicit. These representations are effective for their intended analysis tasks, but they lack the infrastructure for general-purpose optimization or recompilation.

Decompilers also rely on internal IRs to abstract away machine-level details and reduce per-architecture engineering effort. Binary Ninja [23] uses a multi-layer IL system [24]. Ghidra [25] uses P-Code [26] and radare2 [27] uses ESIL, a stack-based IR for lightweight analysis [28]. IDA Pro [29] uses Hex-Rays microcode [30, 31]. These IRs are tailored to decompilation and static analysis, and similarly lack tooling for optimization or recompilation to binary.

LLVM IR occupies a unique position. It originates from a production compiler, carries rich semantic information (e.g., SSA form, type annotations, explicit control flow), and comes with a full optimization and compilation toolchain. Furthermore, optimization passes in LLVM IR perform the same transformations used during normal compilation, making them directly applicable to further re-optimize the IR. No other IR we surveyed offers this combination of semantic expressiveness, mature tooling, and recompilation support. For these reasons, we adopt LLVM IR as the unified representation throughout this work.

3.2. LLVM IR Lifters

Several tools exist for lifting binary code to LLVM IR, and we surveyed the majority of them. Liu et al. [32] categorize their output into two classes. *Emulation-style IR* (EIR) faithfully emulates each machine instruction in LLVM IR, producing code that can execute natively but is verbose and difficult to analyze statically. Lifters in this category include McSema [33] and Rev.ng [34]. *High-level IR* (HIR) refines the emulation-style output through additional recovery passes, aiming for more concise, readable code that is closer to what a compiler would originally emit.

Among HIR lifters, `llvm-mctoll` [35] produces readable and executable LLVM IR, but its architecture support is uneven. It handles x86 well yet fails to lift most ARM binaries in our dataset. RetDec [36] supports a broader set of architectures and achieves the highest IR quality in the benchmark in Liu et al. [32]'s survey. It also successfully lifts over half of the binaries in our dataset. However, closer inspection revealed that RetDec sometimes produces malformed IR and silently emits empty function bodies without

reporting an error. Moreover, all of the lifters mentioned above suffer from significant bugs and have seen little maintenance or updates in recent years.

These limitations led us to a different strategy. Rather than relying on a standalone lifter, we reuse the decompilation frontend of IDA Pro, which is currently the most powerful binary analysis platform in terms of architecture coverage and analysis quality [37]. IDA Pro lifts binaries from diverse architectures into its own unified Hex-Rays microcode. Based on this, we translate the microcode onward into LLVM IR, effectively treating IDA as a multi-architecture lifting frontend without the need to develop or maintain per-ISA conversion logic ourselves. This choice is an engineering one rather than a methodological dependency. An open-source frontend such as Ghidra's P-code [25] could play the same role, and He et al. [3] build a P-code-based lifter for their own IR. Porting our pipeline to such a frontend is straightforward because the underlying idea is identical.

3.3. Related Works

BFSD has increasingly shifted from hand-crafted features toward learned representations. Marcelli et al. [2] provide a comprehensive comparative study of recent ML-based BFSD methods and serve as an important reference point for our work. Existing learning-based approaches span several representation choices. Xu et al. [9] model control-flow graphs with graph neural networks, while Ding et al. [10] and Li et al. [11] learn directly from assembly sequences. Yang et al. [38] move one step higher and learn from Abstract Syntax Tree (AST) style program representations. These methods have achieved clear improvements over traditional matching techniques, but most still treat cross-architecture and cross-optimization variation as something the model itself must absorb. As a result, they have to learn over substantial syntactic and structural noise introduced by different ISAs, compilers, and optimization settings.

Another line of work attempts to reduce this variance before or during analysis. Lifting and re-optimization have been explored in binary rewriting and binary translation [39, 40]. In the context of similarity analysis, David et al. [41] use LLVM IR canonicalization to improve binary matching, but their normalization mainly focuses on simplifying local operations and does not address broader control-structure differences. Altinay et al. [42] also combine lifting and re-optimization, but to recover and optimize programs from *dynamic* execution traces, a different problem from the static, cross-architecture matching we target. More recently, HermesSim [3] lifts binaries into their own IR and constructs semantic oriented graphs (SOGs) enriched with control-flow and data-dependence information, showing that partial normalization can substantially improve performance. Sha et al. [43] further study optimization-aware matching by recovering inlining effects, but their approach targets function inlining specifically rather than the wider space of optimization-induced discrepancies. Zhou et al. [44] lifts and re-optimizes binaries before matching, but it operates on decompiled pseudo-C and relies on generic `-O3` re-optimization. As a short tool demonstration, it does not systematically analyze how these choices affect similarity. Inspired by this, we systematically analyze the variation caused by compilation options, train our model directly on the IR, and build a complete semantic normalization pipeline that also benefits legacy methods.

4. Methodology

In this section, we present the overall methodology of our approach. As illustrated in Figure 1, we first describe the semantic normalization pipeline that lifts binaries into LLVM IR and reduces optimization-induced variance through re-optimization, and then introduce our graph-aware IR model we used to learn function representations.

4.1. Overview

Existing BFSD approaches typically absorb the variations introduced by different instruction set architectures (ISAs) and optimization options into representation learning [2–4, 6, 11, 15, 45], relying on increasingly larger models. As mentioned earlier, we instead mitigate these discrepancies before learning so that the model can focus on the residual differences.

To this end, we design a three-stage pipeline. First, we lift binary functions from diverse architectures into a unified LLVM IR, thereby reducing the syntactic discrepancies caused by different architectures. Second, to address the semantic discrepancies induced by various optimization options, we apply IR-level re-optimization. Finally, based on the optimized IR, we extract Control-Flow Graphs (CFGs) and Data-Dependence Graphs (DDGs) and train a graph-aware language model to produce function embeddings that summarize each function for downstream tasks. Furthermore, because many legacy BFSD baselines accept only raw binaries rather than IR, our pipeline includes an extra binary regeneration phase so that legacy methods can also benefit from the normalization pipeline. The remainder of this section describes each component in detail.

4.2. Semantic Normalization Pipeline

4.2.1. Microcode-to-LLVM Lifting. Motivated by the observations in Section 2, we first lift binary functions from diverse architectures into a unified LLVM IR [16], thereby systematically and deterministically mitigating a class of architecture-induced discrepancies. Among the various IR formats, we select LLVM IR for the reasons discussed in Section 3.1.

For the lifting stage, we do not pursue the path of developing a dedicated lifter for each target architecture, as such an approach requires prohibitive engineering cost and scales poorly for new architectures. Instead, we build our lifting pipeline on top of IDA Pro microcode. Hence, microcode serves as a unified intermediate bridge between

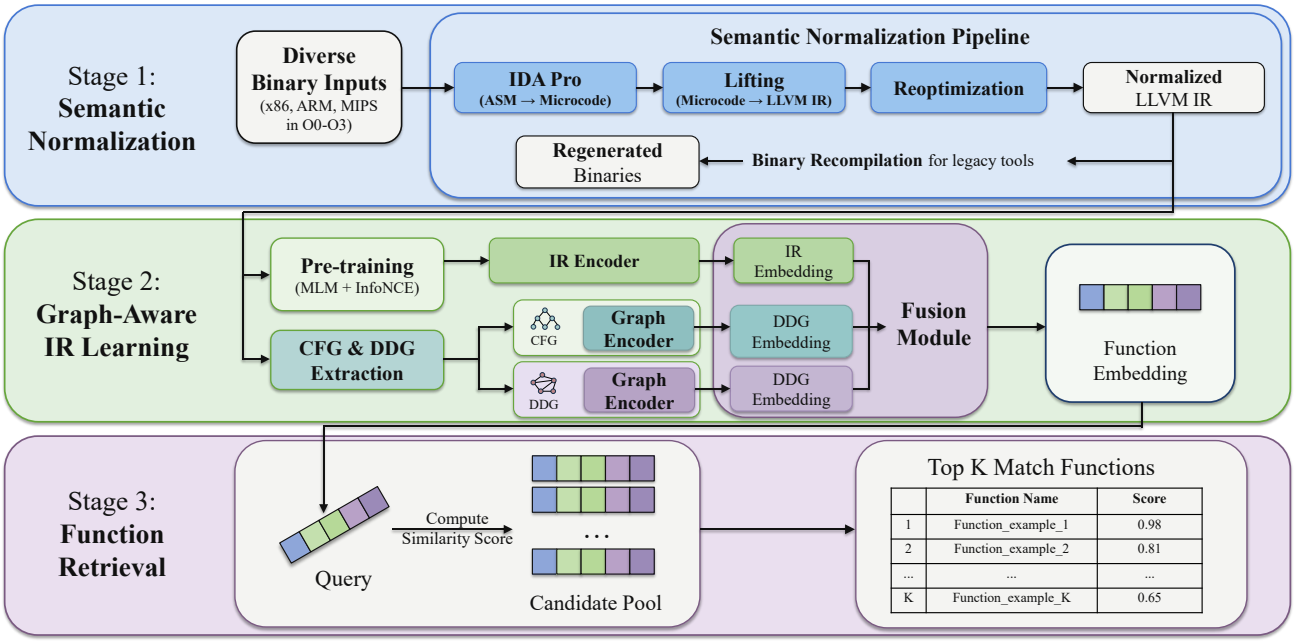


Figure 1: Overview of our proposed framework. The pipeline consists of three main stages: (1) Semantic Normalization mitigates cross-architecture and cross-optimization variances by lifting diverse binaries into a unified normalized LLVM IR. (2) Graph-Aware IR Learning generates a robust function embedding by fusing sequential IR token representations with structural features extracted from CFGs and DDGs via GAT encoders. (3) Function Retrieval identifies the Top-K matching functions by computing vector similarity scores between the query and a candidate pool.

architecture-specific machine code and LLVM IR, enabling us to normalize binaries from different ISAs into a common representation prior to subsequent re-optimization and learning.

4.2.2. IR-level Re-optimization. Beyond architectural differences, another factor that significantly shapes the form of a binary function is compiler optimization. Binaries produced at different optimization levels, or with different optimization flags, may exhibit significant differences even when compiled from the same source code. We aim to mitigate such optimizer-induced discrepancies through an additional normalization pass.

To this end, we introduce an IR-level re-optimization, which we frame as a practical normalization heuristic. We study the organization of common optimization levels in GCC [18] and Clang [17], the two predominant C/C++ compilers. We observe that `-O0` performs essentially no optimization, `-O1` and `-O2` correspond to progressively stronger optimization sets, and `-O3` introduces more aggressive optimizations on top of the preceding levels. More importantly, optimization exhibits an empirically cumulative character. Applying the same or a weaker optimization pipeline to an IR that has already undergone a higher-level optimization typically yields minimal additional change, whereas applying a stronger optimization (e.g., `-O3`) can further simplify or reorganize the code structure. Based on this observation, we hypothesize that lifting functions to LLVM IR and then applying optimization again can reduce part of the divergence introduced by the original compilation configuration, even when the functions were originally compiled at different optimization levels.

We emphasize that we do not claim this pipeline yields a perfect canonical form or absolutely guarantees convergence to identical IR. Given the inherent information loss during compilation and the intricacy of binary files, re-optimization can only remove a subset of compilation-induced discrepancies rather than producing identical code. For the residual discrepancies that cannot be fully eliminated by our pipeline, we introduce our IR-based approach in the remainder of this section.

4.2.3. Binary Regeneration for Legacy Methods. Although LLVM IR is better suited as a unified representation for downstream learning models, a substantial body of existing BFSD methods and established baselines accept only binary files as input only. To enable these legacy tools to benefit from our pipeline, we further recompile the re-optimized LLVM IR back into binary files, which allows us to evaluate the effect of our normalization pipeline across diverse existing methods.

We note that the goal of this recompilation step is not to recover a fully functional executable, but rather to generate binary artifacts consumable by legacy static analysis tools. Because the lifted IR originates from a binary recovery process, it typically lacks the necessary linkage information.

Consequently, we can guarantee a high success rate only at the compilation stage, not at the linking or execution stage. Nonetheless, individual functions remain amenable to function-level symbolic analysis in isolation, which does not require a fully linked, runnable whole-program executable. We exploit this property to validate semantic preservation in Section 6.2. The regenerated binaries produced in this stage are used exclusively for evaluating legacy static analysis methods.

4.3. Graph-Aware Language Model

4.3.1. IR-native Language Model. After the semantic normalization described above, a portion of the discrepancies attributable to architecture and optimization level has been mitigated. However, residual differences remain that cannot be fully resolved (a detailed example is provided in Appendix A). Hence, a model capable of matching functions while tolerating such residual semantic differences on normalized IR is still necessary.

Recent BFSD models predominantly adopt transformer architectures [4, 6, 11, 15], which scale better than pure graph models for large-corpus pretraining and large-pool retrieval [6, 46, 47]. We therefore use a transformer-based model as the base encoder. We train it directly on the normalized LLVM IR rather than on assembly, since lowering back to assembly would add a step and lose the richer structural and semantic information IR retains. As the sequence encoder we adopt `RoFormer` [48], whose rotary position embeddings encode relative rather than absolute positions. This is well suited to LLVM IR, where the relative order of instructions and value propagation is often more informative than absolute position.

Even with normalized input, a sequence model may overfit to token patterns or dataset biases without genuinely capturing the core function semantics [49, 50]. We therefore incorporate program graphs as an auxiliary inductive bias to help the model capture function semantics more explicitly.

4.3.2. Graph-based Augmentation. From the LLVM IR, we directly extract the DDG and the CFG for each function. Compared with the IR sequence alone, the DDG more directly captures value flow and dependency propagation, while the CFG describes the control-transfer structure at the basic-block level. These two categories of graph information serve as important complements to the IR sequence, enabling the model to better capture program semantics [51, 52].

Concretely, we encode a function's LLVM IR token sequence with `RoFormer` [48] and construct separate graph branches over the extracted DDG and CFG using graph neural networks. We then fuse the `RoFormer` and graph embeddings into a final per-function embedding used for retrieval. This adds structure-aware bias while preserving the scalability of the transformer.

4.3.3. Learning Objective. We train the model with two complementary objectives. The first is masked language modeling (MLM). We hide a random subset of the LLVM IR tokens and ask the model to predict them, much like a fill-in-the-blank exercise. To do this well, the model must learn the grammar and meaning of the IR rather than memorize surface text. Prior work finds MLM to be the only effective pretraining signal in this setting [46]. The second objective is contrastive retrieval, which trains the model directly for the retrieval task. For a query function, the model learns to assign high similarity to its true match, the same function compiled under different settings, and low similarity to a large pool of unrelated functions. This mirrors real-world BFSD, where the goal is to find the one matching function among many candidates. We train with both objectives jointly. Details are given in Appendix B, and the full implementation in Section 5.2.

5. Implementation

In this section, we detail the implementation choices of the semantic normalization pipeline and the ReGraphv2 model.

5.1. Semantic Normalization Pipeline

Based on the considerations discussed in Section 3.1, we build LLVM IR on top of IDA Pro's Hex-Rays microcode. We adopt an early open-source prototype as our implementation foundation [53] and apply substantial modifications and refactoring to it. Specifically, we fix a number of bugs in the original implementation, resolve compatibility issues with the latest version of the IDA Pro API, and optimize the overall lifting pipeline for stability and performance. With these modifications, we are able to leverage IDA Pro's existing multi-architecture support to extract LLVM IR from binaries across multiple target architectures, without the need to develop and maintain per-ISA conversion logic as traditional per-architecture lifters require. This property is especially critical for subsequent large-scale data processing and multi-architecture experiments.

After obtaining the lifted IR, we apply a re-optimization pass via LLVM `opt` to achieve semantic normalization. We observed that blindly applying `-O3`, the highest standard optimization level, to re-optimize the lifted IR is not the optimal choice. As detailed in our ablation study (Section 6.7), `-O3` does pull similar functions closer together, but it also shortens the IR so aggressively that dissimilar functions are pushed into a smaller representational space as well, which in turn makes retrieval harder for the model. We therefore study the available optimization passes and, on the validation split, empirically select 12 of them to compose a moderate optimization level, which we call `-Oc` (canonicalization). More details of this choice are provided in Appendix D.

Following normalization, we use `llvm-nm` to enumerate the functions within each IR module and `llvm-extract` to decompose modules into individual per-function IR files, facilitating downstream function-level model training. To accommodate existing tools that accept only binary input, we further recompile each normalized function into a binary

analysis artifact using `clang`. In addition, to support graph-based modeling, we implement custom LLVM passes in C++ that extract the DDG and CFG for each function and export them as graph-structured inputs directly consumable during model training.

5.2. Model and Training Details

On the modeling side, we first train a BPE tokenizer [54] with a vocabulary size of 65,536 on our training corpus to better cover the symbolic patterns commonly present in LLVM IR. We use a `RoFormer` [48] architecture of `base` size as the backbone encoder. We also introduce engineering-level modifications to the attention implementation, replacing the standard attention with memory-efficient attention [55] to support faster training and longer input sequences under constrained hardware resources.

For the graph branches, we employ Graph Attention Networks (GATs) [56] as the encoder for both the DDG and the CFG, with the graph neural network components implemented using the PyG framework [57]. During training, we jointly optimize an MLM loss and a buffered contrastive learning objective [47]. The former enhances the model's understanding of token-level context in LLVM IR, while the latter strengthens discriminability in large-scale candidate-pool retrieval scenarios. More detailed training configurations, software versions, and other related specifics are provided in our code repository[1].

6. Evaluation

We evaluate our proposed approach from five complementary perspectives. First, we examine whether the semantic normalization pipeline is sufficiently robust across diverse binaries and compilation settings. Second, we study how much semantic normalization benefits existing BFSD methods. Third, we compare ReGraphv2 against prior state-of-the-art methods. Fourth, we analyze whether ReGraphv2 achieves a favorable trade-off between accuracy and runtime efficiency in practice. Finally, we assess whether ReGraphv2 is effective in a real-world security setting, namely search for vulnerable functions in firmware images.

- **RQ1:** How robust is the proposed semantic normalization pipeline across diverse binaries and compilation settings?
- **RQ2:** To what extent does semantic normalization improve existing BFSD methods?
- **RQ3:** How does ReGraphv2 compare with prior state-of-the-art methods?
- **RQ4:** Does ReGraphv2 achieve a favorable trade-off between retrieval effectiveness and runtime efficiency in practice?
- **RQ5:** Can ReGraphv2 locate known vulnerable functions in real-world firmware with various architectures?

In this section, we introduce the datasets and metrics used in our evaluation, and then answer the five research questions in turn. We close with an ablation study of the model components and a case study on the role of semantic normalization for LLMs.

TABLE 1: Summary statistics of Dataset-1.

Split	Target Projects	# Bin.	# Func.
Training	OpenSSL, ClamAV, cURL, UnRAR	3,630	847,274
Validation	zlib	940	3,216
Test	Z3, Nmap	919	264,548
Total	7 projects	5,489	1,115,038

6.1. Datasets and Metrics

6.1.1. Coreutils. To obtain an initial assessment of how much the proposed pipeline benefits existing tools, we compile Coreutils [58], a widely used component of Linux systems, for three architectures, namely ARM, MIPS, and x86, each in both 32-bit and 64-bit variants. We further compile each target under `-O0` and `-O3`, which represent the two extreme optimization settings of no optimization and aggressive optimization. Coreutils has been used as a benchmark dataset in prior binary analysis studies [1, 59]. We therefore use it to examine in detail how semantic normalization changes similarity measurements before and after our pipeline.

6.1.2. Dataset-1. A single project is not sufficient for a comprehensive evaluation. We therefore adopt Dataset-1 from [2] as our main benchmark. This dataset covers a broader set of open-source projects and compilation settings, and it has been used in several recent studies in top venues [3, 4, 6, 15]. Using the same benchmark enables direct and fair comparison with prior work while also making our results easier to interpret in the context of the existing literature. Dataset-1 contains seven widely used open-source Linux projects. Each project is compiled with GCC and Clang, with four versions of each compiler, five optimization settings ranging from `-O0` to `-O3` and `-Os`, and three architectures, namely x86, ARM, and MIPS, each in both 32-bit and 64-bit variants. As a result, each program can appear under 240 compilation configurations. We evaluate all methods under the `XM` setting, which requires generalization across architectures, optimization levels, bit widths, compilers, and compiler versions. The dataset is partitioned into training, validation, and test splits with no overlap between them. Table 1 summarizes the dataset statistics. Additional details can be found in [2].

We train the model on the training split and use the validation split for model development. All reported results are computed on the full test split, and all comparisons with prior methods follow the same benchmark setting. During both training and evaluation, we remove all function and variable names to prevent information leakage and shortcut learning.

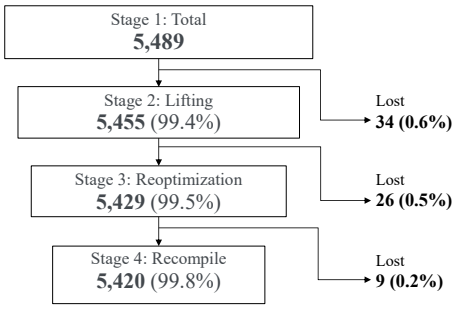


Figure 2: Success rates of the major stages in the semantic normalization pipeline.

6.1.3. Dataset-Vulnerability. We also evaluate on the Dataset-Vulnerability benchmark of Marcelli et al. [2], a real-world security task that searches for known-vulnerable functions across architectures in firmware. The search corpus is the `libcrypto` library taken from two firmware images. One comes from the Netgear R7000 router, compiled for ARM32 with 1,769 candidate functions, and the other from the TP-Link Deco-M4, compiled for MIPS32 with 1,710 candidate functions. The queries are ten known-vulnerable OpenSSL functions that span eight CVEs. We compile each query for x86, x64, ARM32, and MIPS32, search it against the two firmware files, and report MRR@10 as in Marcelli et al. [2]. The firmware libraries come from OpenSSL 1.0.2d built with the vendors' own toolchains, while our training split draws OpenSSL from a much later major version with heavy source-level refactoring, so the exact vulnerable functions are never seen during training.

6.1.4. Recall@1. We use two standard retrieval metrics on Dataset-1, namely Recall@1 and MRR [2]. Recall@1 measures whether the correct match is ranked first in the candidate pool. For each query, the model selects the candidate with the highest similarity score, and the metric is then averaged over all test cases. Recall@1 ranges from 0 to 1. A higher value indicates that the model is more likely to retrieve the correct function as its top prediction.

6.1.5. MRR. Mean Reciprocal Rank measures how highly the correct match is ranked within the candidate pool. For each query, the model assigns similarity scores to all candidates. If the ground truth match is ranked at position p, its reciprocal rank is $\frac{1}{p}$. MRR is the average reciprocal rank over all test cases. A higher MRR indicates that the model places correct matches closer to the top of the ranked list.

6.2. Robustness of the Semantic Normalization Pipeline

As shown in Figure 2, our pipeline successfully lifts, re-optimizes, and recompiles 98.74% of the binaries in Dataset-1. We also achieve a **100% success rate** in Coreutils and Dataset-Vulnerability. We manually inspected the failed cases and found that most of them are caused by corner cases in highly optimized instructions such as SSE and AVX, as well as by invalid IR emitted by the lifter, which then blocks later stages of the pipeline. These observations suggest that the remaining failures are primarily engineering issues rather than fundamental limitations of the approach. Overall, the pipeline is sufficiently robust for large-scale multi-architecture processing.

To further validate that recompilation preserves function semantics, we randomly sample 302 functions from ARM, MIPS, and x86 binaries, covering both 32-bit and 64-bit variants, and perform symbolic equivalence checking. Because symbolic execution is expensive and can be difficult to scale to environment-dependent code, we restrict this check to functions without network communication, file I/O, memory operations, or external function calls. For each sampled function, we compare the original binary function with the function produced after lifting, re-optimization, and recompilation. It is worth noting that the regenerated binaries are not fully linked executables, we perform this comparison on each function in isolation and resolve external dependencies manually, hence, this procedure is for proof of concept purpose and not scalable.

Concretely, we feed both the original binary function f and its recompiled counterpart f' with the same fresh symbolic inputs x, execute them symbolically to obtain the symbolic outputs $f(x)$ and $f'(x)$, and ask a Satisfiability Modulo Theories (SMT) solver called `z3` to determine whether for the same input x, the $f(x) \neq f'(x)$ is satisfiable. If the solver returns unsatisfiable, which means no input can make the two functions diverge, we treat them as equivalent under this check. Otherwise, the satisfying assignment is a concrete counterexample. Across all 302 sampled functions, **the solver finds no counterexample**, indicating that the tested functions preserve their input and output semantics after recompilation.

Answer to RQ1: The proposed semantic normalization pipeline is robust enough for large-scale use. It succeeds for all Coreutils and Dataset-vulnerability binaries and 98.74% of Dataset-1, showing that lifting, re-optimization, and recompilation remain reliable across diverse architectures and compilation settings. In addition, symbolic equivalence checking on sampled functions finds no counterexample, suggesting that recompilation preserves their input-output semantics on the cases tested.

6.3. Effect of Semantic Normalization on Legacy Methods

We first use BinDiff [60] to obtain a fine-grained view of the effect of our pipeline. BinDiff is widely regarded as a standard tool in reverse engineering [1] and has been used in a range of security applications [61, 62]. In this study, we use the BinDiff similarity score to quantify how much semantic normalization increases structural similarity. For

each pair of binaries, BinDiff compares matched functions and assigns a similarity score between 0 and 1 based on graph matching and additional heuristics. We compute the average of these function-level scores per binary file and use it as the binary-level similarity score. Higher scores indicate greater similarity.

Table 2 shows that the pipeline increases the average BinDiff score by 39% across all settings. ARM benefits the most, whereas MIPS shows smaller gains. One reason is that the baseline similarity for MIPS is already relatively high in several settings, which leaves less room for improvement. We also observe a 3% decrease for the comparison between MIPS32 `-O0` and MIPS64 `-O0`. A plausible explanation is that the architectural gap between MIPS32 and MIPS64 is more difficult to normalize than in x86 or ARM. In x86 and ARM, 32-bit registers map more naturally onto portions of their 64-bit counterparts (e.g., EAX register in x86 denotes the lower 32 bits of RAX register). In contrast, MIPS often requires additional sign-extension logic and other adjustments during conversion. Even so, the overall trend remains strongly positive. The remaining gap to a perfect similarity score reflects residual differences that are not completely removed by semantic normalization alone.

We further apply our pipeline to a representative set of baselines from [2] and [3], as summarized in Table 3. We include two classical fuzzy-hash approaches, namely Catalog1 [63] and FSS [64], in order to evaluate the effect on traditional methods. We also include GMN [45] and GNN [9], which are among the strongest graph-based models reported in [2]. Finally, we include Asm2Vec [10], which models assembly as a token sequence and is therefore conceptually closer to our text-based IR encoder. Following the evaluation protocol in [2], we use a candidate pool size of 100 and report Recall@1 and MRR for all the methods.

Table 3 shows that semantic normalization helps every baseline. The biggest gains go to Catalog1 [63] and Asm2Vec [10]. These methods compare functions by their raw bytes and assembly, so they are very sensitive to how a function is compiled. By removing much of the difference that architectures and compilers introduce, our pipeline makes these surface patterns line up across binaries, and the methods that rely on them gain the most. As a result, methods once thought unable to match functions across architectures [2] become far more competitive. FSS [64] also improves a lot, though less, because it hashes only the control-flow structure and still misses differences that lie elsewhere. HermesSim [3] shows smaller but consistent gains, because it already lifts the code and represents it as a semantic graph, which leaves our pipeline less to remove.

Even with these improvements, a clear gap remains between the normalized baselines and our method. This suggests that semantic normalization alone is not sufficient for BFSD. The remaining challenge lies in learning representations that can effectively capture the residual semantic differences after normalization, which is exactly the design goal of ReGraphv2.

Answer to RQ2: Semantic normalization consistently improves all evaluated baselines, raising the average BinDiff similarity score by 39% and benefiting fuzzy-hash and assembly-based methods the most. This indicates that a significant part of the difficulty in BFSD comes from compilation-induced variance that can be reduced before model learning.

6.4. Comparison with Representative Prior Models

Wang et al. [6] show that BFSD performance can degrade substantially as the candidate pool becomes larger. We therefore compare our model with prior state-of-the-art methods across different pool sizes (From 2 to 10,000) using Recall@1 and MRR. Specifically, our baselines include three representation families. For graph-based models, we include GMN [45], the strongest baseline reported by Marcelli et al. [2], and HermesSim [3], which reports strong performance in recent BFSD benchmarks [12–14]. For assembly-based models, we include Trex [65], CLAP [4], whose architecture is closely related to ours but operates directly on assembly, and the embedding LLM Qwen (Qwen3-0.6B-Embedding) [66, 67]. For decompiler-based models, we include Asteria [38], which learns from decompiler-generated ASTs, and DeJina [12], which pairs a code embedding model with decompiled pseudo-C and ranks among the strongest methods, rivaling or even exceeding HermesSim, in the large-scale study from Shi et al. [12]. For fairness, we evaluate these baselines using their original implementations and official weights, reusing published results where available [2, 3]. This subsection thus compares complete systems as published, each on its own intended input. We discuss the improvement for each part of our framework in Section 6.7.

We include Qwen [66] to represent the growing use of LLMs for code and retrieval tasks [4, 12, 68]. Those popular general-purpose LLMs, such as ChatGPT [69] and Claude [70], however, are ill-suited to our setting, where retrieval must scale to thousands or millions of functions and per-query inference cost becomes prohibitive, so we use a compact embedding LLM instead. A detailed discussion of why we exclude general-purpose LLMs is deferred to Appendix C.

Figure 3 shows that our model is the most robust to candidate pool size, and its advantage becomes more pronounced at the large pool sizes that matter for real-world retrieval. The graph-based GMN [45] drops sharply and falls below 0.2 at the largest pools, while the HermesSim [3] degrades more gracefully. Among assembly-based models, Qwen [66] remains a strong baseline even on raw assembly but still falls near 0.5 Recall@1 at pool size 10,000, whereas CLAP [4] and Trex [65] perform poorly, likely because they were originally trained under a more limited architectural diversity. Decompilation acts as a partial normalizer, which makes the decompiler-based methods competitive at small pools. Among them, DeJina [12], a code embedding model on decompiled pseudo-C, performs on par with our model

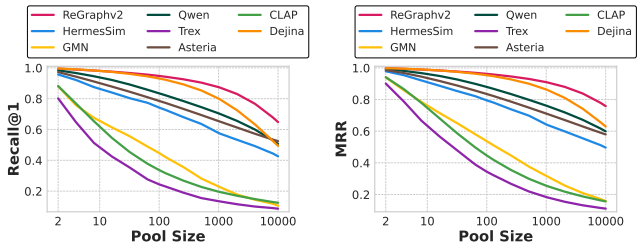
TABLE 2: BinDiff similarity scores before and after lifting, re-optimization, and recompilation.

		O3																				
		ARM32			ARM64			MIPS32			MIPS64			x86			x64			AVG		
		Before	After	Inc	Before	After	Inc	Before	After	Inc	Before	After	Inc	Before	After	Inc	Before	After	Inc	Before	After	Inc
O0	ARM32	0.466	0.681	46%	0.373	0.553	48%	0.362	0.593	64%	0.370	0.519	40%	0.398	0.618	55%	0.408	0.574	41%	0.396	0.590	49%
	ARM64	0.362	0.493	36%	0.474	0.648	37%	0.378	0.547	45%	0.393	0.597	52%	0.426	0.545	28%	0.434	0.630	45%	0.411	0.577	41%
	MIPS32	0.340	0.550	62%	0.374	0.558	49%	0.703	0.754	7%	0.680	0.659	-3%	0.411	0.570	39%	0.422	0.564	33%	0.488	0.609	31%
	MIPS64	0.339	0.535	58%	0.380	0.611	61%	0.665	0.683	3%	0.699	0.753	8%	0.415	0.585	41%	0.426	0.630	48%	0.487	0.633	36%
	x86	0.380	0.593	56%	0.410	0.588	43%	0.410	0.599	46%	0.427	0.563	32%	0.500	0.699	40%	0.465	0.596	28%	0.432	0.606	41%
	x64	0.381	0.539	41%	0.419	0.622	49%	0.404	0.572	42%	0.423	0.588	39%	0.448	0.561	25%	0.508	0.657	29%	0.431	0.590	37%
	AVG	0.378	0.565	50%	0.405	0.597	48%	0.487	0.625	34%	0.499	0.613	28%	0.433	0.596	38%	0.444	0.608	37%	0.441	0.601	39%

TABLE 3: Recall@1 and MRR of baseline models before and after semantic normalization under pool size 100.

Model	Before	After	Delta
HermesSim	0.74 / 0.79	**0.78 / 0.83**	4.85% / 4.66%
GMN	0.45 / 0.53	0.72 / 0.74	60.00% / 39.62%
GNN	0.44 / 0.52	0.66 / 0.74	50.00% / 42.31%
Asm2Vec	0.07 / 0.12	0.27 / 0.35	285.71% / 191.67%
Catalog1	0.09 / 0.26	0.76 / 0.82	**744.44% / 215.38%**
FSS	0.20 / 0.26	0.33 / 0.41	65.00% / 57.69%
ReGraphv2	-	0.93 / 0.95	- / -

Figure 3: Recall@1 and MRR of ReGraphv2 and prior models across different candidate pool sizes.



below pool size 100, with differences small enough to fall within experimental variation. As the pool grows, however, DeJina drops to 0.49 Recall@1 at pool size 10,000. Asteria [38], a Tree-LSTM on decompiler ASTs, trails our model even at small pools and falls further behind as the pool grows. Our IR-level normalization and IR-native modeling extend this small-pool advantage to the large pools that matter, where our method leads every baseline and the gap widens with pool size. Our method also leads in MRR.

Answer to RQ3: ReGraphv2 is the most robust to candidate pool size and leads every baseline at the large pools that matter for real-world retrieval, with the gap widening as the pool grows. Combining semantic normalization with an IR-native model thus scales better than prior approaches.

6.5. Runtime and Efficiency Analysis

To answer **RQ4**, we study two questions. First, whether ReGraphv2 offers a favorable trade-off between retrieval accuracy and inference speed relative to other learning-based models. Second, whether the extra preprocessing introduced by semantic normalization, namely lifting and re-optimization, adds only acceptable overhead rather than dominating the runtime, since a pipeline whose normalization cost far exceeds model inference would be impractical in deployment.

TABLE 4: Parameter counts, model inference latency for processing 100 functions, and accuracy performance under a candidate pool of size 10,000. Lower latency is better while higher Recall@1 and MRR are better.

Model	#. Params	Latency (s)	Recall@1 / MRR
ReGraphv2	148M	2.458	**0.648 / 0.758**
Qwen	596M	9.978	0.507 / 0.599
DeJina	161M	3.767	0.494 / 0.629
CLAP	112M	3.188	0.124 / 0.156
HermesSim	390K	**0.035**	0.426 / 0.496
GMN	**60.5K**	0.876	0.105 / 0.159

We evaluate efficiency under the same hardware setting as prior work [2]. All experiments are conducted on an Ubuntu 24.04 LTS machine with an AMD Ryzen 9950X CPU and an NVIDIA RTX 2080 Ti GPU. Unless otherwise noted, all learning-based models are executed on the same GPU model. Table 4 reports the parameter count, model inference latency for processing 100 functions, and retrieval performance of each model. Since different models use different processing pipelines, we measure only the machine-learning-part latency in this experiment.

As shown in Table 4, ReGraphv2 achieves the best retrieval accuracy by a large margin, reaching 0.648 Recall@1 and 0.758 MRR, and it is also the fastest among the embedding-based models, running about four times faster than Qwen and ahead of CLAP and DeJina. The lightweight graph models HermesSim and GMN attain much lower latency, but at a large cost in accuracy. Overall, ReGraphv2 offers the best accuracy while remaining efficient among comparable learning-based models.

Parameter count alone is not a reliable predictor of runtime, because what a model computes actually matters more than how many parameters it stores. GMN shows one reason. It has far fewer parameters than HermesSim yet runs about 25 times slower, because it compares functions through cross-graph matching, whose irregular computation uses the GPU poorly, whereas HermesSim relies on well-

TABLE 5: Latency breakdown of the proposed pipeline for processing 100 functions under serial and parallel execution. Share denotes the percentage of total runtime contributed by each stage.

Step	Serial		Parallel	
	Latency (s)	Share	Latency (s)	Share
Lifting	1.011	27.65%	0.191	7.06%
Re-optimization	0.187	5.11%	0.055	2.03%
Inference	2.458	67.23%	2.458	90.90%
Total	3.656	100.00%	2.704	100.00%

optimized GPU operations. ReGraphv2 shows another reason. It has more parameters than CLAP but runs faster, because most of its parameters sit in a token lookup table, which is large but adds almost no computation per function.

To further characterize end-to-end cost, Table 5 presents a stage-wise latency breakdown for processing 100 functions under both serial and parallel execution. Since semantic normalization is naturally parallelizable across functions, we use the parallel setting in our default pipeline. In the serial setting, lifting accounts for 27.65% of the runtime. After parallelization, however, inference becomes the dominant bottleneck, while lifting and re-optimization together contribute only 9.10% of the total runtime.

Our lifting stage is a research prototype and is far less optimized than the mature `clang` and LLVM infrastructure [17] that performs the re-optimization. Its overhead therefore reflects the immaturity of our implementation rather than a fundamental cost, and reducing it is an engineering task that lies outside the scope of this proof-of-concept work.

Answer to RQ4: ReGraphv2 provides a practical trade-off between retrieval effectiveness and runtime. It substantially outperforms the compared baselines in retrieval quality while running about four times faster than the Qwen baseline. Although smaller baselines such as HermesSim achieve lower latency in absolute terms, they do so with a clear loss in retrieval effectiveness. Under the default parallel pipeline, preprocessing overhead is modest relative to model inference.

6.6. Real-World Vulnerability Search

To answer **RQ5**, we use the Dataset-Vulnerability benchmark of Marcelli et al. [2], a widely used setup for locating known-vulnerable functions inside real firmware. For each firmware, the candidate pool is its entire `libcrypto` library, which holds 1,769 functions for Netgear R7000 and 1,710 for TP-Link Deco-M4. We compile each known-vulnerable function to x86, x64, ARM32, and MIPS32 and use each version as a separate query against this pool. Because the Netgear image is ARM32 and the TP-Link image is MIPS32, queries from the other architectures are genuine cross-architecture searches. Each architecture column in Table 6 reports MRR@10 for queries compiled to that architecture, following Marcelli et al. [2].

Table 6 reports the full comparison. ReGraphv2 matches or exceeds the strongest baseline GMN on seven of the eight firmware-architecture cells, and it is the only model with a perfect MRR@10 of 1.00 across all four query architectures on Netgear R7000. The lone exception is the x64 query on TP-Link Deco-M4, where GMN reaches 0.73 against our 0.62. GMN is a pairwise graph matcher that does not scale to large pools, whereas ReGraphv2 produces one embedding per function and retrieves by vector similarity. The recovery counts make the gap concrete. Across the 52 vulnerable-function searches, ReGraphv2 ranks the correct function first in 42 and is the only method to recover all 52 within the top 50. GMN follows at 35 and 50, CLAP reaches 22 and 29, and the pseudo-C model DeJina reaches 20 and 48. On Netgear R7000 ReGraphv2 ranks every vulnerable function first, so its high MRR@10 reflects genuine top ranks rather than the low rankings that Marcelli et al. [2] caution MRR@10 can mask.

Answer to RQ5: ReGraphv2 is effective for finding known-vulnerable functions across architectures in real firmware. It matches or outperforms our baseline models on seven out of the eight firmware-and-architecture combinations, and on the Netgear R7000 image it ranks the correct vulnerable function first in every case. This indicates that semantic normalization carries over from our controlled benchmark to real firmware, where locating a vulnerable function across architectures is a practical security need.

6.7. Ablation Study

In this ablation, we measure how much the two graph components of our model, the CFG and the DDG, add on top of the IR. We compare four variants, namely a RoFormer encoder over the IR only, the IR with the CFG, the IR with the DDG, and the IR with both the CFG and the DDG. All variants use the same experimental setting as above. To further isolate the value of the IR itself, we also train a RoFormer model on assembly only. The results are reported in Table 7.

Averaging over 10 random seeds, the full model that combines the IR with both the CFG and the DDG is the strongest variant, improving Recall@1 by 1.42% and MRR by 1.53% over the IR-only encoder. This gain is about six times the seed-to-seed standard deviation, so it is a stable effect rather than an artifact of random initialization. The two graph signals are complementary. The CFG alone gives a small but reliable gain, and the DDG alone stays within seed noise, yet adding the DDG on top of the CFG improves the model further, which is why we keep both. Most of the benefit still comes from the normalized IR itself, and the two graphs add a smaller structural signal on top of it. Our

TABLE 6: Cross-architecture vulnerability search on Dataset-Vulnerability, following the protocol of Marcelli et al. [2]. Per-architecture cells are MRR@10. Top-1 (T1) and Top-50 (T50) count how many vulnerable functions can be found within the top 1 and top 50 retrieval results. Netgear R7000 contributes 16 vulnerable functions and TP-Link Deco-M4 contributes 36, for 52 in total. Avg MRR is the macro-average over the eight per-architecture cells.

	Netgear R7000 (ARM32)						**TP-Link Deco-M4 (MIPS32)**						**Overall**		
Model	x86	x64	ARM32	MIPS32	T1	T50	x86	x64	ARM32	MIPS32	T1	T50	AvgMRR	T1	T50
GMN	0.88	0.54	**1.00**	0.79	12	**16**	0.67	**0.73**	0.70	0.78	23	34	0.76	35	50
GNN	0.33	0.32	0.56	0.30	5	14	0.49	0.56	0.36	0.61	14	34	0.44	19	48
Trex	0.40	0.48	0.44	0.50	5	**16**	0.29	0.42	0.22	0.61	10	31	0.42	15	47
Asm2Vec	0.25	0.13	0.88	0.50	6	12	0.11	0.11	0.02	0.67	8	11	0.33	14	23
CLAP	0.50	0.75	**1.00**	0.28	10	11	0.11	0.22	0.25	0.83	12	18	0.49	22	29
DeJina	0.54	0.63	0.54	0.81	9	15	0.44	0.33	0.50	0.47	11	33	0.53	20	48
ReGraphv2	**1.00**	**1.00**	**1.00**	**1.00**	**16**	**16**	**0.74**	0.62	**0.85**	**1.00**	**26**	**36**	**0.90**	**42**	**52**

TABLE 7: Recall@1 and MRR at pool size 10,000 for different model variants. Delta is measured relative to the IR-only model. The four IR-based variants are averaged over 10 random seeds (mean $\pm$ sample standard deviation). The assembly-only model is a single run.

Variant	**Recall@1**	**MRR**	**Delta**
IR	0.6391 ± 0.0014	0.7467 ± 0.0009	– / –
IR + DDG	0.6400 ± 0.0006	0.7480 ± 0.0005	$+0.14\%$ / $+0.17\%$
IR + CFG	0.6422 ± 0.0013	0.7522 ± 0.0006	$+0.49\%$ / $+0.74\%$
IR + CFG + DDG	$\mathbf{0.6482 \pm 0.0011}$	$\mathbf{0.7581 \pm 0.0007}$	$\mathbf{+1.42\%}$ / $\mathbf{+1.53\%}$
Assembly only	0.3366	0.4142	-47.33% / -44.53%

TABLE 8: Recall@1 and MRR for different optimization variants. Delta is measured relative to the -O3 baseline.

Variant	**Recall@1 / MRR**	**Delta**
O0	0.6038 / 0.7135	+18.32% / +15.40%
O1	0.5001 / 0.6126	-2.00% / -0.92%
O2	0.5063 / 0.6176	-0.78% / -0.11%
O3	0.5103 / 0.6183	- / -
Og	0.5031 / 0.6123	-1.41% / -0.97%
Os	0.5185 / 0.6290	+1.61% / +1.73%
Oc	0.6482 / 0.7581	+27.02% / +22.61%

graph branch is only a single GNN layer and serves as a proof of concept that structure helps rather than as a tuned component, so scaling it is left to future work. We report results over multiple seeds here because the differences are small. Elsewhere in the paper the gaps exceed this variance by an order of magnitude, so single runs are sufficient.

The comparison between IR and assembly is even more revealing. The assembly-only model suffers a dramatic performance drop, which indicates that the benefit of our approach does not stem mainly from the graph-aware architecture alone. Rather, it depends heavily on semantic normalization and on using IR as a more informative representation for downstream retrieval.

We also study how the choice of re-optimization level shapes the normalized IR. The results are in Table 8, measured relative to `-O3`. Surprisingly, the most aggressive standard level `-O3` is not the best, and `-O1`, `-O2`, and `-Og` are even worse than `-O0`, which applies no re-optimization at all. The reason is that aggressive optimization makes functions that should match look alike, but it also makes genuinely different functions look alike, so the model can benefit less from tell matching functions from non-matching ones. We therefore design a moderate level we call `-Oc`, described in Section 5.2 and Appendix D, which removes the differences that compilation introduces while keeping distinct functions easy to tell apart. As Table 8 shows, `-Oc` outperforms `-O3` by 27.02% in Recall@1 and 22.61% in MRR.

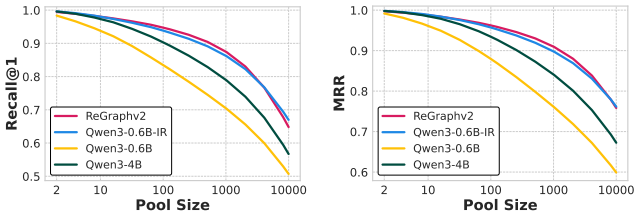
6.8. Case Study: Do We Really Need a Larger Model?

We examine how much semantic normalization can boost an existing model, and in particular whether it lets a smaller model match or even surpass a substantially larger one. We use the Qwen3 embedding family [66] as a controlled comparison because it provides multiple model sizes under a similar training framework [67].

Specifically, we compare Qwen3-0.6B-Embedding, which is the same Qwen baseline used in Section 6, with Qwen3-4B-Embedding. The latter is the largest variant that we can run under our available GPU setting. Both models are first evaluated directly on assembly code, which reflects the most straightforward way of applying a general-purpose embedding model to BFSD without further training. As shown in Figure 4, Qwen3-4B outperforms Qwen3-0.6B in this setting, which is expected given its larger capacity. However, both assembly-based Qwen variants remain substantially below ReGraphv2, which operates on normalized IR. This result suggests that model scale alone does not compensate for the large representation gap induced by architectures and compiler options.

To test this directly, we fine-tune the smaller Qwen3-0.6B on our normalized IR, using the same data and training setting as ReGraphv2. On a candidate pool of 10,000, this 0.6B model on normalized IR outperforms the 4B model on assembly by 18.11% in Recall@1 and 12.25% in MRR,

Figure 4: Recall@1 and MRR of ReGraphv2 and Qwen3 series models across different candidate pool sizes.



despite having roughly one seventh of the parameters. This setup also separates two levers for improving retrieval. The first is normalization, which we measure by switching the input from assembly to normalized IR while keeping the model at 0.6B. The second is scale, which we measure by enlarging the model from 0.6B to 4B while keeping the input as assembly. Both help, but normalization gives the larger gain, which is why a 0.6B model on normalized IR can surpass a model nearly seven times its size on raw assembly.

7. Discussion

Our goal is not to build the best-performing BFSD model but rather to study semantic normalization as a practical design principle and to evaluate our IR-based model as a proof of concept. We acknowledge that larger datasets, longer training, and larger models would likely yield further gains, but pursuing maximal accuracy is outside the scope of this study and is not our main objective. In this section, we discuss the resulting threats to validity and the directions for future work.

7.1. Threats to Validity

One threat to validity comes from function inlining. Dataset-1 is constructed without inlining because our task is defined at function granularity, whereas inlining merges callees into callers and thus alters the function boundary that underlies the ground truth. For the same reason, we disable inlining in our re-optimization pipeline. Addressing inlining is an important challenge for real-world retrieval [5, 43, 71], but it remains orthogonal to our primary goal of studying semantic normalization at the basic function level.

A second threat concerns the robustness of the lifting frontend. Our pipeline currently relies on IDA Pro, meaning any analysis error made by the lifter can propagate to the generated IR and regenerated binaries. Furthermore, adversarial transformations such as code obfuscation, self-modifying code, and binary packing can substantially alter binary contents and break the lifting process entirely. While our empirical results demonstrate strong robustness on standard benchmark binaries, handling adversarially transformed binaries falls outside the scope of the present study.

A final threat concerns dataset scale and model capacity. We deliberately evaluate our lightweight 148M-parameter model on Dataset-1 with 1.1 million functions rather than on massive uncurated datasets or billion-parameter LLMs. Dataset-1 offers an exhaustive, controlled matrix of architectures, compilers, and optimization settings, which is essential for isolating the impact of semantic normalization from the memorization typical of massive models. A compact model also keeps our approach deployable for analysts constrained by local hardware. Scaling to billions of functions or massive LLMs may yield further insights, but our setup already establishes the core point, namely that reducing compilation-induced variance before learning brings substantial gains without relying on maximal model scale.

7.2. Future works

An important direction is to address function inlining more explicitly. Although inlining breaks function-level ground truth by merging callees into callers, it does not eliminate higher-level calling relationships in the call graph. Incorporating call-graph context or inlining-aware matching may extend our approach beyond the current function-level setting. Another direction is to improve the lifting and normalization pipeline itself, which is not yet optimal in efficiency or robustness, especially for highly optimized instructions. Beyond our current `-Oc` heuristic, a more task-aware normalization strategy that selects or adapts optimization passes specifically for BFSD is an important topic for future investigation.

8. Conclusion

In this paper, we showed that much of the difficulty in BFSD stems not from program semantics but from surface differences introduced by architectures, compilers, and optimization levels, and that a substantial part of this variation can be removed before learning. To this end, we proposed a semantic normalization pipeline that lifts binaries into LLVM IR, re-optimizes the IR, and serves both our IR-native model and existing binary-based tools. Our experiments showed that this normalization consistently benefits fuzzy-hash, sequence-based, and graph-based methods, and that our compact model, despite being far smaller than the LLM baselines, retrieves accurately at large scale and recovers real cross-architecture vulnerabilities in real-world firmware. These results suggest that, rather than relying on ever-larger models to absorb every compiler and architecture induced difference, future systems should first remove the variation that can be eliminated in a principled way and then devote model capacity to the semantics that truly distinguish functions. We hope that our study encourages the community to treat the normalization of binary functions as a first-class step in BFSD, so that models can concentrate on what genuinely separates one function from another.

References

[1] Y. Duan, X. Li, J. Wang, H. Yin *et al.*, "Deepbindiff: Learning program-wide code representations for binary diffing," 2020.

[2] A. Marcelli, M. Graziano, X. Ugarte-Pedrero, Y. Fratantonio, M. Mansouri, and D. Balzarotti, "How machine learning is solving the binary function similarity problem," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2099–2116.

[3] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, "Code is not natural language: Unlock the power of {Semantics-Oriented} graph representation for binary code similarity detection," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1759–1776.

[4] H. Wang, Z. Gao, C. Zhang, Z. Sha, M. Sun, Y. Zhou, W. Zhu, W. Sun, H. Qiu, and X. Xiao, "Clap: Learning transferable binary code representations with natural language supervision," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 503–515.

[5] A. Jia, M. Fan, X. Xu, W. Jin, H. Wang, and T. Liu, "Cross-inlining binary function similarity detection," in *Proceedings of the IEEE/ACM 46th international conference on software engineering*, 2024, pp. 1–13.

[6] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "Jtrans: Jump-aware transformer for binary code similarity detection," in *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*, 2022, pp. 1–13.

[7] X. Ren, M. Ho, J. Ming, Y. Lei, and L. Li, "Unleashing the hidden power of compiler optimization on binary code difference: an empirical study," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021, pp. 142–157.

[8] X. Ren, M. Ren, Y. Lei, and J. Ming, "Revisiting optimization-resilience claims in binary diffing tools: Insights from llvm peephole optimization analysis," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2689–2711, 2025.

[9] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 363–376.

[10] S. H. Ding, B. C. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *2019 ieee symposium on security and privacy (sp)*. IEEE, 2019, pp. 472–489.

[11] X. Li, Y. Qu, and H. Yin, "Palmtree: Learning an assembly language model for instruction embedding," in *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, 2021, pp. 3236–3251.

[12] J. Shi, Y. Chen, Y. Xiao, Y. Li, Z. Xu, S. Qiu, C. Zhang, K. Qi, Y. Li, X. Chen *et al.*, "A large scale study of ai-based binary function similarity detection techniques for security researchers and practitioners," *arXiv preprint arXiv:2511.01180*, 2025.

[13] S. Valenzuela and J. Kinder, "Pretraining on call graphs: When binary analysis tasks profit from context," 2026.

[14] Y. Fan, J. Y. Won, D. Zhu, M. Sirlanci, M. Khalili, and C. Yagemann, "Exhib: A benchmark for realistic and diverse evaluation of function similarity in the wild," *arXiv preprint arXiv:2604.01554*, 2026.

[15] Z. Luo, P. Wang, B. Wang, Y. Tang, W. Xie, X. Zhou, D. Liu, and K. Lu, "Vulhawk: Cross-architecture vulnerability detection with entropy-based binary code search." in *NDSS*, 2023.

[16] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," in *International symposium on code generation and optimization, 2004. CGO 2004.* IEEE, 2004, pp. 75–86.

[17] C. Lattner, "Llvm and clang: Next generation compiler technology," in *The BSD conference*, vol. 5, 2008, pp. 1–20.

[18] Free Software Foundation, *Using the GNU Compiler Collection (For GCC version 15.1.0)*, Free Software Foundation, Oxford, UK, 2026. [Online]. Available: https://gcc.gnu.org/onlinedocs/

[19] J. Merrill, "Generic and gimple: A new tree representation for entire functions," in *Proceedings of the 2003 GCC Summit*, 2003, pp. 171–180.

[20] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis," in *IEEE Symposium on Security and Privacy*, 2016.

[21] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proceedings of the 2007 ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '07. ACM, 2007, pp. 89–100.

[22] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "Bap: A binary analysis platform," in *International Conference on Computer Aided Verification*. Springer, 2011, pp. 463–469.

[23] Vector 35 Inc., *Binary Ninja*, Vector 35 Inc., Melbourne, FL, 2026. [Online]. Available: https://binary.ninja/

[24] Vector 35, *BNIL Guide: Overview*, Binary Ninja User Documentation, 2026, binary Ninja Intermediate Language (BNIL) documentation. [Online]. Available: https://docs.binary.ninja/dev/bnil-overview.html

[25] National Security Agency, "Ghidra: A software reverse engineering (SRE) framework," https://github.com/NationalSecurityAgency/ghidra, 2026, version 11.x, Accessed: 2026-04-16.

[26] N. Naus, F. Verbeek, D. Walker, and B. Ravindran, "A

formal semantics for p-code," in *Working Conference on Verified Software: Theories, Tools, and Experiments*. Springer, 2022, pp. 111–128.

[27] Radare2 Project Team, *The Radare2 Book*, 2026, official documentation for the radare2 reverse engineering framework. [Online]. Available: https://book.radare.org/

[28] S. Àlvarez, "radare2: UNIX-like reverse engineering framework and command-line toolset," https://github.com/radareorg/radare2, 2026, version 5.9.x, Accessed: 2026-04-16.

[29] Hex-Rays SA, *IDA Pro (Version 9.0)*, Hex-Rays SA, Liège, Belgium, 2026. [Online]. Available: https://hex-rays.com/ida-pro/

[30] I. Guilfanov, "Decompiler internals: Microcode," in *Black Hat USA*, 2018, hex-Rays decompiler internals white paper / conference briefing.

[31] Hex-Rays, *ida_hexrays — IDAPython References Documentation*, Hex-Rays Documentation, 2026, documents Hex-Rays decompiler internal representations, including microcode and ctree. [Online]. Available: https://python.docs.hex-rays.com/ida_hexrays/index.html

[32] Z. Liu, Y. Yuan, S. Wang, and Y. Bao, "Sok: Demystifying binary lifters through the lens of downstream applications," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1100–1119.

[33] Trail of Bits, "McSema: Framework for lifting x86, aarch64, and ppc64le binary code to LLVM bitcode," https://github.com/lifting-bits/mcsema, 2026, accessed: 2026-04-16.

[34] A. Di Federico, M. Payer, and G. Agosta, "rev. ng: a unified binary analysis framework to recover cfgs and function boundaries," in *Proceedings of the 26th International Conference on Compiler Construction*, 2017, pp. 131–141.

[35] S. B. Yadavalli and A. Smith, "Raising binaries to llvm ir with mctoll (wip paper)," in *Proceedings of the 20th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, ser. LCTES 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 213–218. [Online]. Available: https://doi.org/10.1145/3316482.3326354

[36] J. Křoustek, P. Matula, and P. Zemek, "Retdec: An open-source machine-code decompiler," *July 2018*, 2017.

[37] Y. Cao, R. Zhang, R. Liang, and K. Chen, "Evaluating the effectiveness of decompilers," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 491–502.

[38] S. Yang, C. Dong, Y. Xiao, Y. Cheng, Z. Shi, Z. Li, and L. Sun, "Asteria-pro: Enhancing deep learning-based binary code similarity detection by incorporating domain knowledge," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 1–40, 2023.

[39] D. Ung and C. Cifuentes, "Dynamic binary translation using run-time feedbacks," *Science of Computer Programming*, vol. 60, no. 2, pp. 189–204, 2006.

[40] T. Koju, R. Copeland, M. Kawahito, and M. Ohara, "Re-constructing high-level information for language-specific binary re-optimization," in *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, 2016, pp. 273–283.

[41] Y. David, N. Partush, and E. Yahav, "Similarity of binaries through re-optimization," in *Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation*, 2017, pp. 79–94.

[42] A. Altinay, J. Nash, T. Kroes, P. Rajasekaran, D. Zhou, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, C. Giuffrida *et al.*, "Binrec: dynamic binary lifting and recompilation," in *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020, pp. 1–16.

[43] Z. Sha, Y. Lan, C. Zhang, H. Wang, Z. Gao, B. Zhang, and H. Shu, "Optrans: enhancing binary code similarity detection with function inlining re-optimization," *Empirical Software Engineering*, vol. 30, no. 2, p. 49, 2025.

[44] L. Zhou, M. Dacier, and C. Konstantinou, "Regraph: A tool for binary similarity identification," in *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2025, pp. 6–10.

[45] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *International conference on machine learning*. PMLR, 2019, pp. 3835–3845.

[46] X. Li, L. Gao, S. Yu, Y. Qu, and H. Yin, "On the effectiveness of custom transformers for binary analysis," in *2025 28th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. IEEE, 2025, pp. 411–424.

[47] H. Wang, Z. Gao, C. Zhang, M. Sun, Y. Zhou, H. Qiu, and X. Xiao, "Cebin: A cost-effective framework for large-scale binary code similarity detection," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 149–161.

[48] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[49] R. Saul, C. Liu, N. Fleischmann, R. Zak, K. Micinski, E. Raff, and J. Holt, "Is function similarity over-engineered? building a benchmark," *Advances in Neural Information Processing Systems*, vol. 37, pp. 21 636–21 655, 2024.

[50] X. Xu, S. Feng, Y. Ye, G. Shen, Z. Su, S. Cheng, G. Tao, Q. Shi, Z. Zhang, and X. Zhang, "Improving binary code similarity transformer models by semantics-driven instruction deemphasis," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1106–1118.

[51] Y. Wang, H. Long, L. Zheng, and J. Shang, "Graph-

former: Adaptive graph correlation transformer for multivariate long sequence time series forecasting," *Knowledge-Based Systems*, vol. 285, p. 111321, 2024.
[52] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.
[53] Sandspeare, "ida2llvm: An IDA pro plugin to bridge the gap between IDA's microcode and LLVM IR," https://github.com/Sandspeare/ida2llvm, 2021, accessed: 2026-04-18.
[54] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, 2016, pp. 1715–1725.
[55] M. N. Rabe and C. Staats, "Self-attention does not need $o(n^2)$ memory," *arXiv preprint arXiv:2112.05682*, 2021.
[56] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
[57] M. Fey, J. Sunil, A. Nitta, R. Puri, M. Shah, B. Stojanovič, R. Bendias, A. Barghi, V. Kocijan, Z. Zhang *et al.*, "Pyg 2.0: Scalable learning on real world graphs," *arXiv preprint arXiv:2507.16991*, 2025.
[58] Free Software Foundation, *GNU Coreutils: The GNU Core Utilities Manual*, Free Software Foundation, Boston, MA, 2026, accessed: 2026-04-18. [Online]. Available: https://www.gnu.org/software/coreutils/manual/
[59] F. Zuo, X. Li, P. Young, L. Luo, Q. Zeng, and Z. Zhang, "Neural machine translation inspired binary code similarity comparison beyond function pairs," *arXiv preprint arXiv:1808.04706*, 2018.
[60] Google LLC, "BinDiff: A comparison tool for binary files," https://www.zynamics.com/bindiff.html, 2026, accessed: 2026-04-18.
[61] M. Bourquin, A. King, and E. Robbins, "Binslayer: accurate comparison of binary executables," in *Proceedings of the 2nd ACM SIGPLAN Program Protection and Reverse Engineering Workshop*, 2013, pp. 1–10.
[62] B. Liu, W. Huo, C. Zhang, W. Li, F. Li, A. Piao, and W. Zou, "αdiff: cross-version binary code similarity detection with dnn," in *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*, 2018, pp. 667–678.
[63] xorpd, "The function catalog: A large-scale database of compiled functions," https://www.xorpd.net/pages/fcatalog.html, 2026, accessed: 2026-04-18.
[64] T. Ormandy, "Searching for statically linked vulnerable library functions," Google Project Zero Blog, Dec. 2018, accessed: 2026-04-18. [Online]. Available: https://googleprojectzero.blogspot.com/2018/12/searching-statically-linked-vulnerable.html
[65] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning execution semantics from micro-traces for binary similarity," *arXiv preprint arXiv:2012.08680*, 2020.
[66] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," *arXiv preprint arXiv:2506.05176*, 2025.
[67] Hugging Face, "Text embeddings inference: A blazing fast toolkit for deploying and serving open source text embeddings models," https://github.com/huggingface/text-embeddings-inference, 2026, accessed: 2026-04-20.
[68] Z. Liu, Q. Tang, S. Nie, S. Wu, L. F. Zhang, and Y. Tang, "Keenhash: Hashing programs into function-aware embeddings for large-scale binary code similarity analysis," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 801–824, 2025.
[69] T. Wu, S. He, J. Liu, S. Sun, K. Liu, Q.-L. Han, and Y. Tang, "A brief overview of chatgpt: The history, status quo and potential future development," *IEEE/-CAA Journal of Automatica Sinica*, vol. 10, no. 5, pp. 1122–1136, 2023.
[70] Anthropic, "Claude," 2024. [Online]. Available: https://www.anthropic.com/claude
[71] A. Jia, M. Fan, W. Jin, X. Xu, Z. Zhou, Q. Tang, S. Nie, S. Wu, and T. Liu, "1-to-1 or 1-to-n? investigating the effect of function inlining on binary similarity analysis," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–26, 2023.
[72] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

Appendix A.
A Case Where Normalization Remains Incomplete

This appendix presents a representative case in which our normalization pipeline cannot fully remove compilation-induced discrepancies. Consider the same source-level example discussed in Section 2, but compiled for MIPS32 with `-O1`. After lifting and re-optimization, the resulting program behavior remains semantically equivalent, yet the transformed code does not collapse to the same simplified form produced for the x86 `-O1` version.

Code 7: Pseudo-C for the example function, MIPS32 -O1.

```
int example (int a1, int a2)
{
    int v2;
    if (global_var_1000 > 0)
    {
        v2 = 0;
        do
        {
            a1 += v2;
            ++v2;
        }
        while (v2 < global_var_1000);
    }
    return a1 + (a2 ^ 4);
```

```
}
```

In this MIPS32 case, the loop bound is materialized through a global variable rather than an immediate constant. As a result, during re-optimization, the compiler cannot safely fold the loop using only local function information. This example shows that our semantic normalization pipeline cannot eliminate all discrepancies, which is why we still need the downstream model to learn the remaining semantics from the IR.

Appendix B. Model Hyperparameters and Training Details

This appendix gives additional implementation details for the proposed IR-native model. The model combines a transformer-based sequence encoder with a lightweight graph encoder so that it can use both token-level context and structural information in the IR. We use RoFormer-base to encode serialized IR tokens and GATv2 to encode the graph representation extracted from the same function.

The transformer captures local and long-range dependencies in the token sequence, while the graph branch provides an explicit view of structural relations that may be harder to recover from a purely sequential input. The projection head maps the resulting representation into the embedding space used for similarity learning.

During training, we combine a contrastive objective with masked language modeling. The MLM objective randomly masks a subset of IR tokens and recovers them via the negative log-likelihood in Equation 1, where $p(\cdot)$ is the distribution over the IR vocabulary produced by the model's classification layer, $\mathcal{M}$ is the set of masked indices, T is the input token sequence, and $T_{\setminus i}$ is the visible sequence used to predict the original token t_i at masked position i:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log p(t_i \mid T_{\setminus i}) \tag{1}$$

The contrastive objective trains the model to rank a query function's true match (the same function under different compilation settings) above a large pool of non-matching candidates. We instantiate it as a buffered contrastive (InfoNCE) objective [72] with a momentum-updated negative queue, following CeBin [47]; the exact settings are listed in Table 9 and our repository.

The overall training objective is the sum of the two, jointly optimizing the encoder to be both sensitive to IR syntax and discriminative for retrieval:

$$\mathcal{L}_{\text{all}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{InfoNCE}} \tag{2}$$

The main architectural hyperparameters are summarized in Table 9.

For optimization, we use AdamW with a learning rate of 5×10^{-5} and weight decay of 0.01. Training runs for 300,000 steps with 1,000 warmup steps. The per-device batch size is 4, and gradient checkpointing is enabled to reduce memory usage. We use bfloat16 precision for efficiency.

TABLE 9: Model Architecture Hyperparameters

Category	Hyperparameter	Value
Transformer Encoder	Hidden size	768
	Number of layers	12
	Attention heads	12
	Intermediate (FFN) size	3072
	Max sequence length	2048
Graph Branches (GATv2)	Graph hidden size	768
	Number of GATv2 layers	1
	Graph attention heads	4
	Graph dropout	0.1
Projection Head	Embedding size	768
	Normalization	L2
MoCo	Queue size	65,536
	Momentum	0.999
	Temperature τ	0.07
MLM	Mask probability	0.15

TABLE 10: Optimizer and Training Configurations

Hyperparameter	Value
Optimizer	AdamW (8-bit paged)
Learning rate	5×10^{-5}
Weight decay	0.01
Warmup steps	1,000
Total training steps	300,000
Batch size (per device)	4
Precision	bfloat16
Gradient checkpointing	Enabled

Overall, the configuration is intentionally compact. Instead of relying on a much larger model to absorb compilation-induced variability, we first reduce that variability through semantic normalization and then learn on top of the normalized IR.

Appendix C. Why Not Large General-Purpose LLMs?

Recent advancements in LLM and code analysis have led to the proliferation of massive general-purpose LLMs with hundreds of billions of parameters. A natural question is why we do not simply rely on these massive LLMs to bypass the discrepancies and perform BFSD directly.

The primary reason lies in the structural nature of the BFSD task. Real-world BFSD is inherently a large-scale retrieval problem which is 1-to-N matching rather than a simple pairwise classification task. Our approach, along with prior embedding-based methods, generates offline vector representations for millions of candidate functions. At query

time, finding the match requires only a highly optimized cosine similarity search, which takes milliseconds.

In contrast, massive generative LLMs do not produce dense, comparable embeddings by default. To use them for BFSD, one would either have to perform a pairwise prompt-based comparison (i.e., asking the LLM to compare the query against each of the 10,000 candidates individually), resulting in a catastrophic $O(N)$ inference cost per query, or stuff all candidates into a massively expanded context window, which exceeds current context window limits and suffers from severe attention degradation. Therefore, while massive LLMs possess formidable semantic capacity, they are architecturally and economically infeasible for large-scale binary function retrieval. Our pipeline demonstrates that coupling semantic normalization with a compact, dedicated embedding model (148M parameters) remains the most practical and efficient paradigm for this domain.

Appendix D. Optimization Option Choices

Since there are more than 100 optimization levels, it is infeasible to enumerate all the combinations. For example, in our study, we prefer the optimizations that do not change the function boundaries, hence we disabled all the inline-related optimizations. Also, since running speed is not our primary concern, we disabled all the extreme performance optimizations such as loop unrolling. In total, we roughly categorize the optimizations into the categories demonstrated in Table 11.

TABLE 11: Compiler Optimization Categories and Selected Passes

Category	Selected Passes	Justification
Memory/SSA	`sroa`, `mem2reg`	Employed moderate optimizations to balance performance improvements with code analyzability.
CFG Cleanup	`simplifycfg`, `jump-threading`	Applied strictly to streamline control flow graphs without aggressively restructuring the underlying logic.
Scalar Simplification	`early-cse`, `sccp`, ...	Empirically selected to efficiently reduce scalar operations and propagate constants.
Dead Code Elimination	`dce`, `bdce`, `adce`	Restricted to intra-procedural dead code elimination to isolate function-level effects.
Loop Performance	None	Omitted, as these optimizations significantly alter target loop structures and introduce confounding variables.
Inter-function	None	Omitted to maintain the study's strict focus on function-level (intra-procedural) characteristics.

More aggressive optimization is not always better, since it also pulls distinct functions into a smaller representational space and thereby increases the difficulty for the model to distinguish functions. For instance, we apply one additional optimization, `instcombine`, which combines several instructions into a single simplified instruction, on top of `-Oc`. However, the performance dropped by 2.61%/1.52% in Recall@1/MRR under pool size 10,000.